

به نام خدا



طراحی و پیاده‌سازی یک پایشگر هوشمند ETL مبتنی بر LLM
برای پاکسازی و یکپارچه‌سازی داده‌های جدولی

اعضای گروه: مهسا محمدی و آیناز حامدی

استاد درس: سرکار خانم دکتر طاهائی

نیمسال تحصیلی ۱۴۰۴_۱۴۰۵

فهرست مطالب

2	فهرست مطالب
4	۱. مقدمه
5	۲. بیان مسئله
5	۲.۱ چرا این مسئله اهمیت دارد؟
5	۲.۲ چرا این مسئله از منظر MLOps اهمیت دارد؟
6	۲.۳ سناریوی کاربردی
7	۳. اهداف پروژه
7	۳.۱ هدف اصلی
7	۳.۲ اهداف اختصاصی
8	۴. معماری کلی سیستم
8	۵. توضیح مفهومی و الگوریتمی اجزای سیستم
8	۵.۱ دریافت و پروفایلینگ داده
9	۵.۲ لایه‌ی اعتبارسنجی مبتنی بر قانون
9	۵.۳ موتور تصمیم‌گیری هیبرید (Hybrid Decision Engine)
10	۵.۴ معماری چندعامله (Multi-Agent Architecture)
10	۵.۴.۱ عامل پروفایلر (Agent Profiler)
11	۵.۴.۲ عامل پاکسازی (Agent Cleaner)
11	۵.۴.۳ عامل ناظر (Agent Reviewer)
12	۵.۴.۴ جریان تعامل عامل‌ها
13	۵.۵ لایه‌ی تبدیل و یکپارچه‌سازی خروجی
13	۶. زیرساخت MLOps
13	۶.۱ ردیابی آزمایش با MLflow
14	۶.۲ تشخیص رانش داده (Data Drift Detection)
15	۶.۳ مشاهده‌پذیری، ممیزی‌پذیری و بازتولیدپذیری
16	۷. مجموعه داده‌های ارزیابی
16	۸. روش‌شناسی و معیارهای ارزیابی
17	۹. نتایج تجربی
17	۹.۱ نتایج لایه‌ی قانون‌محور
18	۹.۲ نتایج معماری چندعامله

18	۹.۳ نتایج تشخیص رانش داده.....
19	۹.۴ ثبت و مشاهده پذیری در MLflow
19	۱۰. تحلیل مصالحه‌ی هزینه‌ی محاسباتی و پوشش (Cost–Coverage Trade-off).....
20	۱۱. جمع‌بندی و کارهای آینده.....
20	۱۱.۱ جمع‌بندی.....
21	۱۱.۲ کارهای آینده.....
21	۱۲. منابع.....

۱. مقدمه

کیفیت داده یکی از عوامل تعیین کننده در موفقیت سامانه‌های داده‌محور و یادگیری ماشین است. مطالعات متعدد نشان داده‌اند که بخش قابل توجهی از زمان و منابع هر پروژه‌ی داده — تا ۶۰ الی ۸۰ درصد — صرف آماده‌سازی، پاکسازی و یکپارچه‌سازی داده می‌شود، پیش از آنکه اصلاً بتوان به تحلیل یا آموزش مدل پرداخت. با وجود این، ابزارهای سنتی فرایند ETL (استخراج، تبدیل، بارگذاری) عمدتاً بر پایه‌ی قوانین از پیش تعریف شده عمل می‌کنند و توانایی محدودی در درک معنایی و زمینه‌ای داده دارند؛ برای نمونه، یک قانون ساده می‌تواند تشخیص دهد که یک سلول خالی است، اما نمی‌تواند بر اساس سایر ستون‌های یک رکورد، مقدار محتمل آن را استنتاج کند.

ظهور مدل‌های زبانی بزرگ (LLM) و توانایی آن‌ها در درک متن، استدلال زمینه‌ای و انجام وظایف ساختاریافته، فرصتی جدید برای غنی‌سازی فرایندهای پاکسازی داده فراهم کرده است. با این حال، بهره‌گیری مستقیم و بی‌قیدوشرط از این مدل‌ها — به‌ویژه از طریق سرویس‌های ابری — با دو چالش اساسی روبه‌رو است: نخست، هزینه‌ی محاسباتی و تأخیر بالای ارسال حجم زیادی از رکوردها به یک مدل زبانی؛ و دوم، ریسک‌های حریم خصوصی هنگام ارسال داده‌های حساس سازمانی به سرویس‌های خارجی. از سوی دیگر، در ادبیات موجود، توجه کافی به جایگاه این فرایندها در چارچوب گسترده‌تر MLOps — یعنی مشاهده‌پذیری، ردیابی آزمایش‌ها، و پایش مستمر کیفیت داده در طول زمان — مبذول نشده است. این پروژه با هدف پر کردن این شکاف، یک چارچوب سبک، ترکیبی (Hybrid) و کاملاً محلی برای پایش و پاکسازی هوشمند داده‌های جدولی طراحی و پیاده‌سازی کرده است. در ادامه‌ی این گزارش، ابتدا مسئله و اهمیت آن از منظر مهندسی داده و MLOps تبیین می‌شود، سپس معماری سیستم و منطق الگوریتمی هر یک از اجزای آن به تفصیل شرح داده می‌شود، و در پایان، روش‌شناسی ارزیابی و نتایج تجربی به‌دست‌آمده از اجرای واقعی سیستم روی چهار مجموعه‌داده‌ی متفاوت ارائه می‌گردد.

۲. بیان مسئله

۲.۱ چرا این مسئله اهمیت دارد؟

در سامانه‌های داده‌محور واقعی، کیفیت پایین داده می‌تواند به‌طور مستقیم به افت دقت مدل‌های یادگیری ماشین، نتایج تحلیلی نادرست و در نهایت تصمیم‌گیری‌های اشتباه سازمانی منجر شود. مشکلات رایج کیفیت داده شامل مقادیر گم‌شده، رکوردهای تکراری، ناسازگاری در فرمت و ساختار، و خطاهای معنایی (مانند مقادیری که از نظر ساختاری معتبرند اما از نظر منطقی نادرستند) است. با وجود گستردگی این مشکلات، اکثر ابزارهای موجود در این حوزه — نظیر *Great Expectations* — تنها قادر به تشخیص خطاهای ساختاری‌اند و از درک معنایی داده عاجزند.

از منظر کاربردی، سه شکاف اصلی در راه‌حل‌های موجود قابل مشاهده است:

- ابزارهای مبتنی بر قانون تنها خطاهای ساختاری (مانند خالی بودن یا عدم تطابق نوع داده) را تشخیص می‌دهند و نسبت به ناسازگاری‌های معنایی — مثلاً مقداری که از نظر منطق کسب‌وکار نامعتبر است — کور هستند.
- راه‌حل‌های مبتنی بر LLM ابری (مانند فراخوانی API مدل‌های تجاری) برای داده‌های حساس سازمانی مناسب نیستند و هزینه‌ی مقیاس‌پذیری آن‌ها بالاست.
- هیچ ابزار یکپارچه‌ای که هم‌زمان پاکسازی هوشمند و مشاهده‌پذیری آن را در چارچوب MLOps ارائه دهد، در مقیاس دانشگاهی یا سازمان‌های کوچک در دسترس نیست.

۲.۲ چرا این مسئله از منظر MLOps اهمیت دارد؟

MLOps را می‌توان مجموعه‌ای از اصول و ابزارهایی تعریف کرد که چرخه‌ی حیات سامانه‌های یادگیری ماشین را — از آماده‌سازی داده تا استقرار و پایش مدل — قابل تکرار، قابل ردیابی و قابل اتوماسیون می‌سازند. سه رکن اصلی MLOps عبارت‌اند از خودکارسازی (Automation)، پایش مستمر (Continuous Monitoring) و بازتولیدپذیری (Reproducibility).

یکی از خلاءهای رایج در پیاده‌سازی‌های عملی MLOps، غفلت از لایه‌ی داده به‌عنوان بخش بالادستی (Upstream) خط لوله است؛ در حالی که بسیاری از مشکلاتی که در نهایت به‌صورت افت عملکرد مدل ظاهر می‌شوند، در واقع ریشه در تغییرات کیفیت یا توزیع داده‌ی ورودی دارند.

این پروژه بر پایه‌ی سه استدلال اصلی در این باره بنا شده است:

- پایش کیفیت داده در زمان اجرا (Runtime Data Quality Monitoring) باید جزء جدایی‌ناپذیر هر سامانه‌ی MLOps سالم باشد، نه یک فعالیت جانبی و دستی.
- رانش داده (Data Drift) که اغلب ریشه در تغییر توزیع ورودی‌های فرایند ETL دارد، باید در همان لایه‌ی ETL — پیش از آنکه به مدل برسد — شناسایی شود، نه پس از استقرار مدل و مشاهده‌ی افت عملکرد آن.
- ثبت رخداد ساختارمند (Structured Logging) در طول فرایند ETL، پیش‌نیاز دو ویژگی حیاتی هر خط لوله‌ی MLOps یعنی بازتولیدپذیری (Reproducibility) و قابلیت ممیزی (Auditability) است.

۲.۳ سناریوی کاربردی

برای روشن‌تر شدن کاربرد عملی این چارچوب، سناریوی زیر را در نظر بگیرید: یک تیم دانشگاهی یا یک استارت‌آپ کوچک در حال ساخت یک سامانه‌ی پیش‌بینی مبتنی بر داده‌های جدولی است. داده‌های ورودی این تیم از چند منبع ناهمگن جمع‌آوری می‌شود، بودجه و زیرساخت محدود است و استفاده از API‌های مدل‌های زبانی ابری هزینه‌بر است، و در نهایت، تیم امکان بررسی دستی و روزانه‌ی کیفیت داده را ندارد. این پروژه دقیقاً برای چنین سناریویی، یک راه‌حل خودکار، کاملاً محلی و قابل‌مشاهده ارائه می‌دهد.

۳. اهداف پروژه

۳.۱ هدف اصلی

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک چارچوب یکپارچه برای پایش هوشمند فرایندهای ETL است که با بهره‌گیری هم‌زمان از روش‌های مبتنی بر قواعد و مدل‌های زبانی بزرگ محلی، کیفیت داده‌های جدولی را بهبود داده و امکان مشاهده‌پذیری کامل فرایند را در چارچوب MLOps فراهم می‌کند.

۳.۲ اهداف اختصاصی

- شناسایی خودکار خطاهای کیفیت داده شامل مقادیر گمشده، داده‌های تکراری، ناسازگاری‌های ساختاری و خطاهای معنایی.
- طراحی و پیاده‌سازی یک موتور مسیریابی هیبرید (Hybrid Router) به‌منظور تصمیم‌گیری هوشمند در تخصیص وظایف میان روش‌های مبتنی بر قانون و مدل‌های زبانی بزرگ، با هدف کاهش هزینه‌ی محاسباتی و زمان پردازش.
- پیاده‌سازی لایه‌ی استنتاج محلی مبتنی بر LLM با استفاده از مدل‌های متن‌باز کوانتیزه‌شده، بدون وابستگی به سرویس‌های ابری یا API‌های تجاری.
- طراحی معماری چندعامله شامل عامل تحلیل‌گر، عامل پاکسازی و عامل ارزیاب برای مدیریت و اعتبارسنجی فرایند پاکسازی داده.
- یکپارچه‌سازی سامانه با ابزارهای MLOps — به‌ویژه MLflow — به‌منظور ثبت، پایش و ردیابی معیارهای کیفیت داده، عملکرد مدل و اطلاعات هر اجرای سیستم.
- ارزیابی کمی عملکرد سیستم از طریق مقایسه‌ی کیفیت داده پیش و پس از پردازش و اندازه‌گیری معیارهایی نظیر کامل‌بودن، سازگاری، نرخ تکرار و امتیاز بهبود کیفیت داده.
- طراحی یک لایه‌ی پایش و مشاهده‌پذیری برای ردیابی تغییرات کیفیت داده، ثبت رخدادها، تحلیل عملکرد عامل‌ها و تشخیص رانش در داده‌های ورودی.
- ارائه‌ی راهکاری سبک‌وزن و قابل‌اجرا در محیط‌های دانشگاهی و سازمانی کوچک، بدون نیاز به زیرساخت‌های پیچیده‌ی آموزش مدل یا سرویس‌های ابری.

۴. معماری کلی سیستم

سیستم پیشنهادی از یک معماری چندلایه تشکیل شده است که هر لایه مسئولیت مشخصی در زنجیره‌ی پردازش داده بر عهده دارد. این لایه‌ها به ترتیب عبارت‌اند از: دریافت داده، پروفایلینگ داده، اعتبارسنجی مبتنی بر قانون، تصمیم‌گیری هیبرید، استنتاج مدل زبانی محلی، اعتبارسنجی خروجی، تبدیل و یکپارچه‌سازی نهایی، و در نهایت پایش و ثبت رخداد. طراحی این معماری به گونه‌ای است که هر لایه مستقل از دیتاست خاص عمل می‌کند و بنابراین کل خط لوله بدون تغییر در منطق اصلی، روی مجموعه داده‌های مختلف با ساختار متفاوت قابل اجراست.

جدول زیر نقش هر لایه را به طور خلاصه نشان می‌دهد:

نقش اصلی	لایه
خواندن فایل خام، تشخیص ساختار و استانداردسازی اولیه	دریافت داده (Ingestion)
محاسبه‌ی آمار توصیفی، تشخیص مقادیر گمشده و رکوردهای تکراری	پروفایلینگ داده (Profiling)
اعمال قواعد قطعی (یکتایی، عدم تهی بودن، بازه‌ی مجاز)	اعتبارسنجی مبتنی بر قانون
مسیریابی هر ستون خطا دار به مسیر قانون محور یا مدل زبانی	موتور تصمیم‌گیری هیبرید
استنتاج معنایی برای پیشنهاد مقدار اصلاح شده	لایه‌ی مدل زبانی محلی
بررسی و پذیرش یا رد پیشنهادهای مدل زبانی	اعتبارسنجی خروجی
ترکیب خروجی دو مسیر و تولید داده‌ی نهایی پاکسازی شده	تبدیل و یکپارچه‌سازی
ثبت تمام تصمیمات، معیارها و مصنوعات (Artifacts) در MLflow	پایش و ثبت رخداد

۵. توضیح مفهومی و الگوریتمی اجزای سیستم

۵.۱ دریافت و پروفایلینگ داده

لایه‌ی دریافت داده مسئول خواندن فایل‌های ورودی (در قالب CSV) و استانداردسازی اولیه‌ی آن‌هاست؛ از جمله تشخیص و یکسان‌سازی مقادیر گمشده‌ای که ممکن است در فرمت‌های متفاوتی مانند علامت سؤال، رشته‌ی خالی یا کلمه‌ی «نامشخص» بیان شده باشند. پس از آن، لایه‌ی پروفایلینگ روی داده‌ی خام اجرا می‌شود و یک گزارش کیفیت اولیه در قالب ساختاریافته تولید می‌کند.

این گزارش شامل درصد مقادیر گم شده به تفکیک هر ستون، شناسایی رکوردهای تکراری از طریق تطابق دقیق، آمار توصیفی (حداقل، حداکثر، میانگین و انحراف معیار برای ستون‌های عددی)، و فهرست ستون‌هایی است که نیازمند پردازش بیشتر تشخیص داده شده‌اند. این گزارش، ورودی مستقیم موتور تصمیم‌گیری در مرحله‌ی بعد است.

۵.۲ لایه‌ی اعتبارسنجی مبتنی بر قانون

در این لایه، مجموعه‌ای از قواعد قطعی و قابل محاسبه در زمان چندجمله‌ای — یعنی با پیچیدگی محاسباتی خطی نسبت به تعداد رکوردها — روی داده اجرا می‌شود. این قواعد شامل بررسی یکتایی مقادیر شناسه‌محور (Unique Constraint)، عدم وجود مقدار تهی در ستون‌های حیاتی، و اعتبارسنجی بازه‌ی مجاز برای ستون‌های عددی است. هدف اصلی این لایه، حل قطعی و کم‌هزینه‌ی خطاهایی است که نیازی به استدلال معنایی ندارند؛ برای مثال، پر کردن یک ستون عددی با میانه‌ی مقادیر موجود، یک تصمیم قانون‌محور و قابل توجیه آماری است که نیازی به فراخوانی مدل زبانی ندارد. کاهش حجم داده‌ای که باید به لایه‌ی پرهزینه‌تر مدل زبانی ارسال شود، دقیقاً هدف طراحی این لایه است.

۵.۳ موتور تصمیم‌گیری هیبرید (Hybrid Decision Engine)

موتور تصمیم‌گیری، نقش مغز کنترلی سیستم را ایفا می‌کند و مسئول تعیین مسیر پردازش برای هر ستون خطا دار است. منطق این موتور بر پایه‌ی دو معیار اصلی استوار است: نوع داده‌ی ستون (عددی یا دسته‌ای) و نسبت کاردینالیتی آن، یعنی نسبت تعداد مقادیر یکتا به تعداد کل مقادیر غیر خالی آن ستون. الگوریتم مسیریابی را می‌توان به صورت زیر خلاصه کرد:

- اگر ستون از نوع عددی باشد، خطا (در این حالت، عمدتاً مقادیر گم شده) با روش قطعی جایگذاری میانه حل می‌شود، زیرا میانه معیاری مقاوم در برابر داده‌های پرت است و نیازی به استدلال معنایی ندارد.
- اگر ستون از نوع دسته‌ای باشد و نسبت کاردینالیتی آن پایین‌تر از یک آستانه‌ی از پیش تعریف شده باشد (یعنی مجموعه‌ی محدودی از مقادیر ممکن دارد)، ستون به عنوان کاندید نیازمند استدلال معنایی شناسایی شده و به مسیر مدل زبانی هدایت می‌شود.

- اگر ستون دسته‌ای باشد اما نسبت کاردینالیتی آن بسیار بالا باشد (یعنی تقریباً هر مقدار آن یکتاست، مانند نام افراد یا شماره‌ی بلیط)، ستون شبه‌شناسه (Identifier-like) در نظر گرفته می‌شود؛ چرا که در چنین ستون‌هایی، استدلال معنایی مبتنی بر نمونه‌های مشابه معنای آماری ندارد، و این ستون به مسیر قانون‌محور (جایگذاری با مقدار پرتکرار یا بازبینی دستی) هدایت می‌شود.

این منطق، پیاده‌سازی مستقیم ادعای «مسیریابی هیبرید» است که در بخش نوآوری این پروژه مطرح شده: به‌جای ارسال بی‌قیدوشرط همه‌ی داده به مدل زبانی، سیستم ابتدا خطاهای ساده را با هزینه‌ی محاسباتی خطی حل می‌کند و تنها داده‌های واقعاً نیازمند استدلال معنایی را به مدل زبانی می‌سپارد. این رویکرد به‌طور مستقیم به کاهش تأخیر (Latency) و هزینه‌ی محاسباتی سیستم منجر می‌شود، که در سناریوهای مقیاس بزرگ اهمیت عملی چشمگیری دارد.

۵.۴ معماری چندعامله (Multi-Agent Architecture)

در این پروژه، مدل زبانی بزرگ صرفاً به‌عنوان یک تابع تولید متن مورد استفاده قرار نمی‌گیرد، بلکه در قالب معماری عامل‌محور مبتنی بر الگوی ReAct (استدلال و اقدام) به کار گرفته می‌شود. این رویکرد به عامل‌ها امکان می‌دهد ضمن استدلال درباره‌ی مسئله، ابزار مناسب را انتخاب کرده و تصمیمات چندمرحله‌ای اتخاذ کنند، به‌گونه‌ای که فرایند پاکسازی و اعتبارسنجی داده به‌صورت هوشمند، قابل‌ردیابی و قابل توضیح اجرا شود. سیستم شامل سه عامل اصلی است که به‌ترتیب زیر با یکدیگر تعامل می‌کنند:

۵.۴.۱ عامل پروفایلر (Agent Profiler)

این عامل مسئول تحلیل اولیه‌ی داده و تولید گزارش کیفیت است. وظایف آن شامل محاسبه‌ی درصد مقادیر گم‌شده به تفکیک ستون، شناسایی رکوردهای تکراری، محاسبه‌ی آمار توصیفی، و تولید یک گزارش ساختاریافته (JSON) شامل شاخص‌های کیفیت داده است. این گزارش، ورودی مستقیم عامل بعدی — یعنی موتور تصمیم‌گیری — است.

۵.۴.۲ عامل پاکسازی (Agent Cleaner)

این عامل، گزارش تولیدشده توسط عامل پروفایلر و تصمیمات موتور تصمیم‌گیری را دریافت می‌کند و برای ستون‌هایی که به مسیر مدل زبانی هدایت شده‌اند، اصلاحات معنایی پیشنهاد می‌دهد. الگوریتم این عامل به‌ازای هر رکورد دارای مقدار گمشده، یک درخواست (Prompt) با تکنیک Few-Shot Prompting می‌سازد که شامل زمینه‌ی سایر ستون‌های همان رکورد و — در صورت محدود بودن مجموعه‌ی مقادیر ممکن — فهرست مقادیر مجاز است. سپس این درخواست از طریق یک سرویس استنتاج محلی (Ollama) به مدل زبانی کوانتیزه‌شده ارسال می‌شود و مدل موظف است پاسخ خود را در قالب یک شیء ساختاریافته شامل مقدار پیشنهادی و یک امتیاز اطمینان (Confidence Score) بین صفر تا صد بازگرداند.

به‌منظور کنترل هزینه‌ی محاسباتی و زمان اجرا — که در سامانه‌های محلی و بدون زیرساخت GPU اختصاصی اهمیت ویژه‌ای دارد — تعداد رکوردهایی که از هر ستون به مدل زبانی ارسال می‌شوند، توسط یک سقف پیکربندی‌پذیر (Max Rows per LLM Batch) محدود می‌شود. این تصمیم طراحی، بخشی جدایی‌ناپذیر از فلسفه‌ی مسیریابی هیبرید سیستم است: به‌جای پردازش کامل و پرهزینه‌ی همه‌ی موارد، سیستم یک نمونه‌ی نماینده را از طریق مدل زبانی پردازش می‌کند و می‌تواند بر اساس نرخ موفقیت آن نمونه، در مورد کیفیت و قابلیت اتکای مدل روی کل مجموعه‌داده قضاوت کند. مقادیر گمشده‌ای که فراتر از این سقف باقی می‌مانند، از طریق یک مکانیزم بازگشتی (Fallback) با مقدار پرتکرار همان ستون (Mode Imputation) تکمیل می‌شوند تا کامل‌بودن نهایی داده تضمین شود، بدون آنکه هزینه‌ی محاسباتی به‌صورت خطی با اندازه‌ی مجموعه‌داده رشد کند.

۵.۴.۳ عامل ناظر (Agent Reviewer)

این عامل مسئول ارزیابی و اعتبارسنجی خروجی تولیدشده توسط عامل پاکسازی است، پیش از آنکه هر تغییری روی داده‌ی نهایی اعمال شود. منطق این عامل بر دو معیار استوار است: نخست، مقایسه‌ی امتیاز اطمینان خودگزارش‌شده‌ی مدل با یک آستانه‌ی از پیش تعیین‌شده — تنها اصلاحاتی که امتیاز اطمینان آن‌ها از این آستانه بالاتر باشد، پذیرفته می‌شوند

دوم، بررسی انطباق مقدار پیشنهادی با قوانین مرجع (Ground Rules) دامنه — به عنوان نمونه، برای ستونی با مجموعه‌ی محدود مقادیر مجاز، مقدار پیشنهادی باید دقیقاً در همان مجموعه قرار گیرد. این مکانیزم دوگانه، نقش یک لایه‌ی دفاعی در برابر توهم‌زایی (Hallucination) مدل زبانی ایفا می‌کند و تضمین می‌کند که تنها اصلاحات قابل‌اتکا وارد داده‌ی نهایی می‌شوند. تمامی تصمیمات این عامل — پذیرفته و رد شده — به‌طور کامل ثبت می‌شوند تا امکان ممیزی و تحلیل عملکرد سیستم فراهم باشد.

۵.۴.۴ جریان تعامل عامل‌ها

فرایند اجرای معماری چندعامله، به‌صورت یک زنجیره‌ی پنج مرحله‌ای و کاملاً قابل‌ردیابی اجرا می‌شود:

- 1 عامل پروفایلر داده را تحلیل کرده و گزارش کیفیت را در قالب ساختاریافته تولید می‌کند.
- 2 موتور تصمیم‌گیری بر اساس نتایج این گزارش، مسیر پردازش مناسب (قانون‌محور یا مدل زبانی) را برای هر ستون خطادار انتخاب می‌کند.
- 3 ستون‌های مسیریابی شده به روش قانون‌محور بلافاصله و به‌صورت قطعی اصلاح می‌شوند.
- 4 برای ستون‌های نیازمند تحلیل معنایی، عامل پاکسازی فعال شده و اصلاحات پیشنهادی را به همراه امتیاز اطمینان تولید می‌کند.
- 5 عامل ناظر خروجی را اعتبارسنجی کرده، تغییرات معتبر را اعمال می‌کند، و تمامی مراحل، تصمیمات و نتایج در MLflow ثبت و ذخیره می‌شوند.

این معماری به‌صورت یک حلقه‌ی دستی از نوع (Thought → Action → Observation) ReAct پیاده‌سازی شده است؛ هر عامل در هر مرحله، دلیل تصمیم خود (Thought)، اقدام انجام‌شده (Action) و نتیجه‌ی مشاهده‌شده (Observation) را ثبت می‌کند. این طراحی، اگرچه از چارچوب‌های آماده‌ای مانند LangChain یا CrewAI استفاده نمی‌کند، اما از نظر مفهومی معادل همان الگوی تعامل عامل‌محور است و هر یک از کلاس‌های عامل، به‌گونه‌ای طراحی شده که در آینده بتوان آن را به‌سادگی به‌عنوان یک ابزار (Tool) در چنین چارچوب‌هایی نیز بسته‌بندی کرد.

۵.۵ لایه‌ی تبدیل و یکپارچه‌سازی خروجی

در نهایت، خروجی مسیر قانون‌محور و خروجی تأییدشده‌ی مسیر مدل زبانی در این لایه با یکدیگر ترکیب می‌شوند تا مجموعه‌داده‌ی نهایی و پاکسازی‌شده تولید شود. این لایه همچنین مسئول اعمال تبدیلات ساختاری نهایی (مانند یکسان‌سازی نوع داده‌ی ستون‌ها) و آماده‌سازی داده برای مصرف در مراحل بعدی — مانند آموزش مدل یا تحلیل — است.

۶. زیرساخت MLOps

در این پروژه، مفاهیم بنیادین MLOps نه به‌عنوان یک افزوده‌ی جانبی، بلکه به‌عنوان یکی از ارکان اصلی طراحی در نظر گرفته شده‌اند. سه مؤلفه‌ی اصلی این زیرساخت عبارت‌اند از ردیابی آزمایش با MLflow، تشخیص رانش داده در لایه‌ی ETL، و لایه‌ی مشاهده‌پذیری و ممیزی‌پذیری که در ادامه هر کدام به تفصیل شرح داده می‌شوند.

۶.۱ ردیابی آزمایش با MLflow

MLflow به‌عنوان بستر اصلی پایش، ردیابی و مشاهده‌پذیری سامانه به کار گرفته شده و به‌طور کامل به‌صورت محلی (بدون نیاز به زیرساخت ابری) اجرا می‌شود. تمامی فعالیت‌های سیستم — از اجرای خط لوله‌ی قانون‌محور گرفته تا اجرای معماری چندعامله و تشخیص رانش — در قالب یک «اجرا» (Run) مجزا ثبت می‌شوند. هر اجرا شامل چهار دسته اطلاعات است:

هدف	متغیرهای ثبت‌شده	دسته
مقایسه‌ی کیفیت داده پیش و پس از پردازش	کامل بودن، نرخ تکرار، سازگاری	کیفیت داده
پایش کارایی و قابلیت اتکای مدل زبانی	تأخیر هر رکورد، امتیاز اطمینان، نرخ پذیرش	کارایی مدل زبانی
شناسایی تغییر توزیع در داده‌های ورودی	آماره‌های آزمون آماری بین دو دسته‌ی داده	تشخیص رانش
قابلیت ممیزی و اشکال‌زدایی	نوع خطا، مسیر انتخابی، اصلاح اعمال‌شده	اقدامات عامل‌ها

این ساختار ثبت، امکان مقایسه‌ی اجراهای مختلف (مثلاً روی مجموعه‌داده‌های متفاوت یا با پیکربندی‌های متفاوت موتور تصمیم‌گیری)، بازتولید دقیق نتایج، و تحلیل روند تغییرات کیفیت داده در طول زمان را فراهم می‌کند — که دقیقاً همان چیزی است که مفهوم Reproducibility در MLOps به آن اشاره دارد.

۶.۲ تشخیص رانش داده (Data Drift Detection)

رانش داده به تغییر تدریجی یا ناگهانی در توزیع آماری داده‌های ورودی گفته می‌شود که می‌تواند عملکرد مدل‌های پایین‌دستی را — بدون آنکه خود مدل تغییر کرده باشد — به شدت تحت تأثیر قرار دهد. در معماری پیشنهادی، تشخیص رانش در همان لایه‌ی ETL و پیش از ورود داده به هر مدل پایین‌دستی انجام می‌شود؛ رویکردی که تشخیص زودهنگام تغییرات را نسبت به شناسایی آن‌ها پس از استقرار و افت مشاهده‌شده‌ی عملکرد مدل، سریع‌تر، کم‌هزینه‌تر و مؤثرتر می‌سازد.

الگوریتم تشخیص رانش بر پایه‌ی مقایسه‌ی آماری دو دسته‌ی داده — یک دسته‌ی مرجع (Reference Batch) و یک دسته‌ی جاری (Current Batch) — عمل می‌کند و بسته به نوع هر ستون، از دو رویکرد مکمل استفاده می‌کند:

- برای ستون‌های عددی، از آزمون ناپارامتریک کولموگوروف-اسمیرنوف (Kolmogorov-Smirnov Test) برای مقایسه‌ی توزیع تجمعی دو نمونه استفاده می‌شود؛ اگر مقدار p این آزمون از یک آستانه‌ی معنی‌داری پایین‌تر باشد، فرض یکسان بودن دو توزیع رد می‌شود. به موازات آن، واگرایی کولبک-لایبیلر (KL Divergence) روی هیستوگرام دو نمونه محاسبه می‌شود تا شدت تغییر توزیع نیز به صورت کمی گزارش شود.
- برای ستون‌های دسته‌ای، از شاخص پایداری جمعیت (Population Stability Index یا PSI) — یکی از رایج‌ترین معیارهای صنعتی برای تشخیص تغییر توزیع دسته‌ای — و واگرایی متقارن جنسن-شانون (Jensen-Shannon Divergence) به عنوان معیار مکمل استفاده می‌شود. عبور شاخص PSI از یک آستانه‌ی مشخص (معمولاً ۰.۲، به عنوان آستانه‌ی متداول «تغییر معنی‌دار جمعیت» در صنعت) به عنوان نشانه‌ی رانش تلقی می‌شود.

نکته‌ی مهم الگوریتمی در طراحی این لایه، لزوم حذف ستون‌های شبه‌شناسه (مانند نام یا شماره‌ی بلیط) از فرایند تشخیص رانش است؛ چرا که شاخص‌هایی مانند PSI بر پایه‌ی مقایسه‌ی فراوانی دسته‌های تکرارشونده تعریف می‌شوند، و در ستونی که تقریباً هر مقدار آن یکتاست، این شاخص از نظر آماری بی‌معنا و بی‌ثبات می‌شود. به همین دلیل، همان معیار نسبت کاردینالیتی که در موتور تصمیم‌گیری هیبرید برای تشخیص ستون‌های شبه‌شناسه به کار رفته، در این لایه نیز برای فیلتر کردن ستون‌های نامناسب برای تحلیل رانش استفاده می‌شود؛ اصلی که پایداری و اعتبار آماری خروجی نهایی را تضمین می‌کند.

برای اعتبارسنجی این الگوریتم، دو نوع سناریوی آزمایشی طراحی شد: یک «آزمون شبیه‌سازی» (Simulation Test) که با تزریق عمدی جابه‌جایی در توزیع ستون‌های عددی و بازوزن‌دهی دسته‌های ستون‌های دسته‌ای، یک دسته‌ی داده‌ی دارای رانش مصنوعی می‌سازد و سیستم باید آن را به‌درستی شناسایی کند؛ و یک «آزمون کنترلی» که با تقسیم تصادفی یک مجموعه‌داده به دو نیمه — بدون هیچ رانش واقعی — بررسی می‌کند که سیستم هشدار کاذب (False Positive) تولید نکند. هر دو سناریو در روش‌شناسی ارزیابی این پروژه به‌عنوان روش «Simulation Test» مطرح شده‌اند.

۶.۳ مشاهده‌پذیری، ممیزی‌پذیری و بازتولیدپذیری

فراتر از ثبت متریک‌های عددی، سیستم به‌گونه‌ای طراحی شده که هر تصمیم اتخاذشده در طول خط لوله — از جمله مسیر انتخابی هر ستون توسط موتور تصمیم‌گیری، متن دقیق درخواست ارسالی به مدل زبانی، پاسخ خام دریافتی، و دلیل پذیرش یا رد هر پیشنهاد توسط عامل ناظر — در قالب فایل‌های ساختاریافته (JSON) ذخیره می‌شود. این سطح از جزئیات، سه ویژگی کلیدی MLOps را تحقق می‌بخشد: نخست، قابلیت ممیزی (Auditability)، یعنی امکان بازبینی دقیق اینکه چرا یک مقدار خاص در داده اصلاح شده است؛ دوم، قابلیت اشکال‌زدایی (Debuggability)، یعنی امکان ردیابی منشأ هر خطا یا رفتار غیرمنتظره تا ریشه‌ی آن در زنجیره‌ی تصمیم‌گیری؛ و سوم، بازتولیدپذیری (Reproducibility)، یعنی امکان اجرای مجدد دقیقاً همان فرایند و دستیابی به نتایجی قابل مقایسه با اجرای قبلی.

۷. مجموعه داده‌های ارزیابی

برای ارزیابی جامع سیستم روی انواع مختلف خطای کیفیت داده، چهار مجموعه داده با ویژگی‌ها و چالش‌های متفاوت انتخاب شدند. دو مجموعه داده (Adult Income و Titanic) از منابع دانشگاهی معتبر و عمومی تهیه شدند. از آنجا که مقوله‌ی «OpenML Dirty Tabular» در واقع مجموعه‌ای از دهه‌ها جدول ناهمگون است و نه یک فایل واحد و ثابت، و مجموعه داده‌ی مصنوعی سبک LLMClean نیز طبق تعریف خود، تولیدی و نه دانلودی است، این دو مجموعه داده از طریق یک تولیدکننده‌ی مصنوعی و بذرمحور (Seeded) بازسازی شدند که دقیقاً همان انواع خطای ذکرشده در پروپوزال پروژه (ناسازگاری ساختاری و خطای معنایی) را به صورت کنترل شده و تکرارپذیر در داده تزریق می‌کند.

خطای اصلی	تعداد رکورد (تقریبی)	منبع	مجموعه داده
مقادیر گم‌شده، عدم تطابق نوع داده	۸۹۱	Kaggle / UCI	Titanic
ناسازگاری، نویز، رکورد تکراری	۳۲,۵۶۱	UCI ML Repository	Adult Income
ناسازگاری ساختاری، خطای معنایی	۲,۰۲۰	تولیدشده با بذر ثابت	OpenML Dirty (مصنوعی)
خطای معنایی، فرمت ترکیبی	۱,۰۰۰	تولیدشده با بذر ثابت	LLMClean (مصنوعی)

۸. روش‌شناسی و معیارهای ارزیابی

ارزیابی سیستم در سه سطح مستقل انجام می‌شود: کیفیت داده، کارایی مدل زبانی، و پایش/مشاهده‌پذیری. جدول زیر معیارها، روش اندازه‌گیری و آستانه‌ی موفقیت هر یک را نشان می‌دهد:

آستانه‌ی موفقیت	روش اندازه‌گیری	معیار	هدف ارزیابی
$\leq 95\%$	مقایسه‌ی قبل/بعد از ETL	Completeness	کیفیت داده
$\leq 90\%$	اعتبارسنجی قانون‌محور	Consistency Score	کیفیت داده
$\geq 1\%$	مقایسه‌ی قبل/بعد	Duplicate Rate	کیفیت داده
> 5 ثانیه بر رکورد	زمان‌سنجی هر رکورد	LLM Latency	کارایی مدل
$\leq 80\%$	مقایسه با برچسب دستی	Error Correction Rate	کارایی مدل

هدف ارزیابی	معیار	روش اندازه‌گیری	آستانه‌ی موفقیت
پایش	Log Coverage	ممیزی لاگ‌ها	٪۱۰۰
پایش	Drift Detection Delay	آزمون شبیه‌سازی	> ۱ دسته‌ی داده

فرمول‌های محاسباتی معیارهای اصلی به شرح زیر تعریف شده‌اند:

$\text{Completeness} = (\text{تعداد مقادیر غیرتهی} \div ۱۰۰) \times \text{کل مقادیر}$ $\text{Duplicate Rate} = (\text{تعداد رکوردهای تکراری} \div ۱۰۰) \times \text{کل رکوردها}$ $\text{Error Correction Rate} = ۱۰۰ \times (\text{کل خطاهای معنایی شناسایی‌شده} \div \text{LLM تعداد خطاهای اصلاح‌شده توسط})$ $\text{Data Quality Improvement Score} = ((\text{امتیاز پس از پردازش} - \text{امتیاز پیش از پردازش}) \div ۱۰۰) \times \text{امتیاز پیش از پردازش}$

نکته‌ی مهم مفهومی در محاسبه‌ی نرخ اصلاح خطا (Error Correction Rate) آن است که مخرج این کسر باید برابر با «کل خطاهای معنایی شناسایی‌شده» توسط موتور تصمیم‌گیری باشد، نه صرفاً تعداد رکوردهایی که در عمل به مدل زبانی ارسال شده‌اند.

۹. نتایج تجربی

تمامی نتایج ارائه‌شده در این بخش، حاصل اجرای واقعی سیستم — شامل فراخوانی مدل زبانی محلی (Mistral-۷B) با کوانتیزیشن ۴ بیتی از طریق Ollama — روی هر چهار مجموعه‌داده است، نه شبیه‌سازی یا برآورد نظری.

۹.۱ نتایج لایه‌ی قانون‌محور

مجموعه‌داده	کامل‌بودن پیش از پردازش	کامل‌بودن پس از پردازش	رکورد تکراری حذف‌شده
Titanic	٪۹۱.۹۰	٪۱۰۰.۰۰	۰
Adult Income	٪۹۹.۱۳	٪۱۰۰.۰۰	۲۴
OpenML Dirty	٪۹۹.۰۹	٪۱۰۰.۰۰	۱۷
LLMClean	٪۹۹.۴۶	٪۱۰۰.۰۰	۰

در هر چهار مورد، لایه‌ی قانون‌محور به‌تنهایی موفق به رساندن کامل بودن داده به ۱۰۰٪ و نرخ تکرار به صفر شد؛ نتیجه‌ای که آستانه‌ی موفقیت تعریف‌شده در بخش ۸ (کامل بودن $\leq 95\%$ و نرخ تکرار $\geq 1\%$) را در هر دو مورد پوشش می‌دهد.

۹.۲ نتایج معماری چندعامله

جدول زیر نتایج اجرای کامل معماری چندعامله را نشان می‌دهد. ستون «خطاهای معنایی شناسایی‌شده» تعداد کل مقادیر گم‌شده در ستون‌های مسیریابی‌شده به مدل زبانی است، ستون «ارسال‌شده به مدل زبانی» تعداد نمونه‌هایی است که واقعاً از طریق مدل زبانی پردازش شده‌اند، و «نرخ اصلاح خطا» طبق فرمول رسمی این پروژه محاسبه شده است.

نرخ اصلاح خطا (فرمول رسمی)	پذیرفته‌شده توسط ناظر	ارسال‌شده به مدل زبانی	خطای معنایی شناسایی‌شده	مجموعه داده
۱۴.۵۱٪	۱۰۰	۱۰۲	۶۸۹	Titanic
۶.۳۸٪	۲۷۲	۳۰۰	۴,۲۶۱	Adult Income
۹۸.۳۳٪	۵۹	۶۰	۶۱	OpenML Dirty
۱۰۰.۰۰٪	۲۰	۲۰	۲۰	LLMClean

در کنار نرخ اصلاح خطای رسمی، معیار مکمل «نرخ پذیرش نمونه‌ی ارسالی» (نسبت پذیرفته‌شده به ارسال‌شده) نیز محاسبه شده است تا کیفیت خالص عملکرد مدل زبانی — مستقل از سیاست محدودسازی حجم — قابل مشاهده باشد. این نرخ برای هر چهار مجموعه داده به ترتیب برابر ۹۸.۰۴٪، ۹۰.۶۷٪، ۹۸.۳۳٪ و ۱۰۰.۰۰٪ به دست آمد.

۹.۳ نتایج تشخیص رانش داده

الگوریتم تشخیص رانش روی هر چهار مجموعه داده از طریق روش «آزمون شبیه‌سازی» اجرا شد. در تمامی موارد، سیستم موفق به شناسایی رانش تزریق‌شده در ستون‌های عددی و دسته‌ای معتبر (پس از حذف ستون‌های شبه‌شناسه) شد و تأخیر تشخیص رانش، طبق طراحی الگوریتم، در همان یک دسته‌ی داده (Batch) رخ داد — که آستانه‌ی موفقیت «کمتر از یک دسته» تعریف‌شده در بخش ۸ را برآورده می‌کند.

در آزمون کنترلی مکمل (تقسیم تصادفی یک مجموعه داده بدون رانش واقعی)، سیستم هیچ هشدار کاذبی روی ستون‌های معتبر (غیرشناسه‌ای) تولید نکرد که اعتبار آماری الگوریتم را تأیید می‌کند.

۹.۴ ثبت و مشاهده‌پذیری در MLflow

تمامی اجراهای فوق — شامل خط لوله‌ی قانون‌محور، معماری چندعامله، و تشخیص رانش — در قالب آزمایش‌های مجزا (Experiments) در MLflow ثبت شدند و از طریق رابط کاربری محلی آن قابل مشاهده، مقایسه و ردیابی هستند. این ثبت شامل پارامترهای اجرا (نظیر سیاست مسیریابی و مدل زبانی استفاده‌شده)، معیارهای کمی، و مصنوعات (Artifacts) شامل گزارش‌های کامل تصمیمات هر عامل است.

۱۰. تحلیل مصالحه‌ی هزینه‌ی محاسباتی و پوشش (Cost-Coverage Trade-off)

نتایج بخش ۹.۲ یک الگوی روشن و از نظر مفهومی قابل توجیه را نشان می‌دهد: در دو مجموعه داده‌ای که تعداد خطاهای معنایی شناسایی شده در محدوده‌ی سقف پردازش مدل زبانی قرار داشت (OpenML Dirty و LLMClean)، نرخ اصلاح خطا به بالای ۹۸٪ رسید — به‌وضوح فراتر از آستانه‌ی موفقیت ۸۰٪. در مقابل، در دو مجموعه داده‌ای که تعداد خطاهای معنایی به‌طور قابل توجهی از این سقف فراتر رفت (Adult Income و Titanic)، نرخ اصلاح خطای رسمی به ترتیب به ۱۴.۵۱٪ و ۶.۳۸٪ رسید.

این الگو، نه یک ضعف تصادفی، بلکه پیامد مستقیم و قابل پیش‌بینی همان اصل «مسیریابی هیبرید» است: سیستم عمداً حجم داده‌ای که به مدل زبانی ارسال می‌شود را محدود می‌کند تا هزینه‌ی محاسباتی و زمان اجرا، متناسب با اندازه‌ی مجموعه داده رشد نکند. شاهد این ادعا آن است که نرخ پذیرش نمونه‌ی ارسالی — یعنی کیفیت خالص مدل زبانی مستقل از حجم — در هر چهار مجموعه داده به‌طور پیوسته بالای ۹۰٪ باقی ماند. به بیان دیگر، افت نرخ اصلاح خطای رسمی در دو مجموعه داده‌ی بزرگ‌تر، نشانه‌ی ضعف مدل زبانی نیست، بلکه نتیجه‌ی مستقیم یک تصمیم طراحی آگاهانه برای کنترل هزینه‌ی محاسباتی است.

این یافته از منظر MLOps اهمیت ویژه‌ای دارد: نشان می‌دهد که سقف پردازش مدل زبانی (Max Rows per LLM Batch) یک ابرپارامتر (Hyperparameter) عملیاتی است که مستقیماً میان دو

هدف رقیب — کیفیت پوشش کامل (Coverage) و کارایی هزینه‌ای (Cost Efficiency) — مصالحه برقرار می‌کند، و مقدار بهینه‌ی آن به منابع محاسباتی در دسترس و حجم داده‌ی مورد انتظار در محیط عملیاتی بستگی دارد. در استقراریهایی که منابع محاسباتی بیشتری در دسترس است یا زمان اجرای طولانی‌تر قابل قبول است، افزایش این سقف می‌تواند نرخ اصلاح خطای رسمی را — با توجه به نرخ پذیرش بالای مشاهده‌شده — به‌طور قابل توجهی افزایش دهد، بدون آنکه نیازی به تغییر در معماری یا منطق الگوریتمی سیستم باشد. این ویژگی، خود گواهی بر انعطاف‌پذیری طراحی سیستم پیشنهادی است.

۱۱. جمع‌بندی و کارهای آینده

۱۱.۱ جمع‌بندی

در این پروژه، یک چارچوب سبک، ترکیبی و کاملاً محلی برای پایش و پاکسازی هوشمند داده‌های جدولی طراحی و پیاده‌سازی شد. این چارچوب با ترکیب روش‌های قطعی مبتنی بر قانون و استدلال معنایی مبتنی بر مدل زبانی محلی، توانست کامل بودن داده را در هر چهار مجموعه داده‌ی مورد آزمایش به ۱۰۰٪ برساند و نرخ تکرار را به صفر کاهش دهد. معماری چندعامله‌ی پیاده‌سازی شده (شامل عامل‌های پروفایلر، پاکسازی و ناظر)، فرایند پاکسازی معنایی را به‌صورت قابل‌ردیابی و قابل توضیح اجرا کرد، و یکپارچگی کامل با MLflow، مشاهده‌پذیری و قابلیت ممیزی سیستم را — دو رکن اساسی هر خط لوله‌ی MLOps سالم — تضمین نمود. علاوه بر این، لایه‌ی مستقل تشخیص رانش داده، امکان شناسایی زود هنگام تغییرات توزیع ورودی را در همان لایه‌ی ETL و پیش از استقرار هر مدل پایین‌دستی فراهم آورد.

تحلیل نتایج نشان داد که نرخ اصلاح خطای رسمی سیستم به‌شدت متأثر از سیاست کنترل هزینه‌ی محاسباتی است؛ یافته‌ای که به‌جای تضعیف اعتبار سیستم، در واقع درک عمیق‌تری از مصالحه‌ی ذاتی میان کیفیت پوشش و کارایی هزینه‌ای در سامانه‌های عملیاتی مبتنی بر مدل زبانی محلی فراهم می‌کند — موضوعی که کمتر در ادبیات موجود به‌صورت کمی و شفاف مورد بحث قرار گرفته است.

۱۱.۲ کارهای آینده

- اندازه‌گیری رسمی معیار Consistency Score از طریق یکپارچه‌سازی کامل لایه‌ی اعتبارسنجی مبتنی بر قانون در خط لوله‌ی عمومی سیستم برای هر چهار مجموعه داده.
- اندازه‌گیری دقیق و آماری معیار LLM Latency per Row در مقیاس بزرگ‌تر و روی سخت‌افزارهای مختلف، برای تدوین یک راهنمای عملی انتخاب سقف پردازش مدل زبانی متناسب با منابع در دسترس.
- مطالعه‌ی تجربی رابطه‌ی میان سقف پردازش مدل زبانی و نرخ اصلاح خطای رسمی، به منظور یافتن نقطه‌ی بهینه‌ی مصالحه‌ی هزینه-پوشش برای سناریوهای مختلف استقرار.
- بسته‌بندی رسمی عامل‌های پیاده‌سازی شده در قالب ابزارهای (Tools) چارچوب‌های استاندارد عامل محور نظیر LangChain یا CrewAI، برای افزایش قابلیت همکاری با اکوسیستم گسترده‌تر ابزارهای عامل محور.
- گسترش استراتژی جایگذاری ستون‌های با نرخ مقادیر گمشده‌ی بسیار بالا (مانند ستون Cabin در مجموعه داده‌ی Titanic) به سمت مهندسی ویژگی جایگزین، نظیر تبدیل به یک متغیر دودویی به جای جایگذاری مستقیم مقدار.

۱۲. منابع

- Zhang et al., "A Survey of LLM-based Data Preparation", ۲۰۲۴ [۱]
- Li et al., "LLMClean: Context-Aware Data Cleaning", ۲۰۲۴ [۲]
- Wang et al., "AutoDCWorkflow", ۲۰۲۴ [۳]
- "Rahm, E., Do, H., "Data Cleaning: Problems and Current Approaches [۴]
- Chen et al., "LLM Agents for Tabular Data Cleaning", ۲۰۲۵ [۵]
- Great Expectations Documentation [۶]

Zhang, W., et al. (۲۰۲۳). "Data-Copilot: Bridging Billions of Data and Humans [۷]
".with Autonomous Workflow

Khattab, O., et al. (۲۰۲۴). "DSPy: Compiling Declarative Language Model [۸]
.Calls into State-of-the-Art Pipelines." ICLR

:Kreuzberger, C., et al. (۲۰۲۳). "Machine Learning Operations (MLOps) [۹]
.Overview, Definition, and Architecture." IEEE Access

Chen, Y., & Wang, H. (۲۰۲۵). "LLMOps: Operationalizing Large Language [۱۰]
.Models in Enterprise and Air-gapped Environments." JDIQ

MLflow Team. (۲۰۲۴). MLflow Documentation. mlflow.org/docs/latest [۱۱]

Ollama. (۲۰۲۴). Ollama — Run LLMs Locally. ollama.ai [۱۲]